

Advanced Machine Learning

DATA 642

Lecture 10 — Other Algorithms for Clustering

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Idea Behind Spectral Clustering

- **Objective:**

- Cluster data points based on the similarity of their feature representations.

- **Methodology:**

1. **Construct Affinity Matrix:**

- Define pairwise similarities or distances between data points.
- Represent data points as vertices in a graph, where edge weights represent similarities.

2. **Compute Graph Laplacian:**

- Transform the affinity matrix into a graph Laplacian matrix.
- Encodes the geometric structure and connectivity of the data.

3. **Eigenvalue Decomposition:**

- Compute the eigenvectors and eigenvalues of the graph Laplacian.
- Eigenvectors capture the embedding of data points in a low-dimensional space.

4. **Cluster Embedding Space:**

- Use the eigenvectors corresponding to the smallest eigenvalues to embed data points into a lower-dimensional space.
- Perform clustering (e.g., K-Means) in the embedded space to identify clusters.

- **Benefits:**

- Captures complex geometric structures and non-linear relationships in the data.
- Effective for clustering datasets with irregular shapes or non-convex clusters.
- Provides a flexible and scalable approach to clustering high-dimensional data.

- **Applications:**

- Image segmentation
- Social network analysis
- Gene expression analysis
- Dimensionality reduction
- Various other domains requiring clustering in high-dimensional spaces.

Basic Math Concepts for Clustering

- **Pairwise Similarity or Distance**

- Before constructing the affinity matrix, pairwise similarities or distances between data points need to be defined. This can be done using various metrics such as Euclidean distance or cosine similarity.
- For two data points $\mathbf{x}_i$ and $\mathbf{x}_j$, the Euclidean distance is calculated as:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2} \quad (1)$$

- The cosine similarity between two vectors $\mathbf{u}$ and $\mathbf{v}$ is given by:

$$\text{cosine similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (2)$$

- **Affinity Matrix** After defining pairwise similarities or distances, they are represented in an affinity matrix A . The elements a_{ij} of the affinity matrix represent the similarity or distance between data points $\mathbf{x}_i$ and $\mathbf{x}_j$.
- **Graph Laplacian** The graph Laplacian matrix L can be computed from the affinity matrix A as $L = D - A$ where D is a diagonal matrix with the degree of each vertex on the diagonal.
- **Eigenvalue Decomposition**
 - After computing the graph Laplacian, eigenvalue decomposition is performed to compute the eigenvectors and eigenvalues of the Laplacian matrix. This step helps in capturing the embedding of data points in a low-dimensional space.

Computing eigenvalues and eigenvectors

1. Direct Methods:

- Direct methods involve computing the eigenvalues directly using techniques such as the QR algorithm, the Jacobi method, or the bisection method. These methods are straightforward but can be computationally expensive, especially for large matrices.

2. Iterative Methods:

- Iterative methods, such as the power iteration method or the Lanczos algorithm, are more efficient for large sparse matrices. These methods iteratively refine an initial estimate of the eigenvalues until convergence is reached.

3. Matrix Decomposition:

- Matrix decomposition techniques, such as the Singular Value Decomposition (SVD) or the Cholesky decomposition, can be used to compute the eigenvalues of the Laplacian matrix indirectly.

4. Graph Partitioning Algorithms:

- Graph partitioning algorithms, such as spectral partitioning or graph bisection, can be used to compute approximate eigenvalues of the Laplacian matrix. These algorithms partition the graph into smaller subgraphs based on the eigenvectors of the Laplacian matrix, allowing for efficient computation of eigenvalues.

5. Sparse Matrix Techniques:

- For large sparse Laplacian matrices, specialized sparse matrix techniques and libraries (e.g., ARPACK) can be used to compute a subset of the eigenvalues and eigenvectors efficiently.